

CO3 – ASSESSMENT TOOL 1: EXPERIMENTS

Name :Konjeti Pavan Deekshith

Reg .no:192412172

Sub Code:ITA0502

Sub Name: Computer Vision

Experiment 1: Feature Descriptor Comparison (SIFT vs Harris)

Aim: To design and perform an experiment comparing Harris Corner Detection and SIFT (Scale Invariant Feature Transform) on a common set of images, and to analyze their performance in terms of feature quality, robustness, repeatability, and suitability for image matching applications.

Procedure:

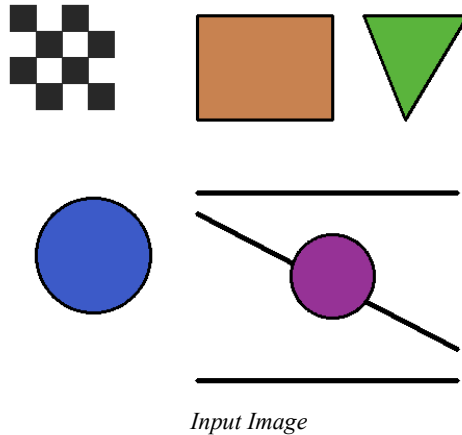
1. Select a set of sample images, including images with variations in rotation, scale, and illumination.
2. Study the theoretical principles of Harris Corner Detection (based on the auto-correlation of image gradients and the second-moment matrix) and SIFT (keypoint detection using Difference of Gaussian and scale-space extrema).
3. Load the selected images using a suitable software/tool such as Python with OpenCV.
4. Apply the Harris Corner Detection algorithm on each image and mark the detected corner points.
5. Apply the SIFT algorithm on the same set of images to detect keypoints and compute their descriptors.
6. Display and systematically document the detected features obtained from both techniques.
7. Apply transformations such as rotation, scaling, and noise addition to the images, and repeat both detection methods to test repeatability.
8. Perform feature matching between image pairs using the SIFT descriptors and evaluate the matching accuracy.
9. Tabulate the results in terms of number of features detected, robustness to transformations, repeatability, and matching accuracy.
10. Analyze and compare the performance of Harris Corner Detection and SIFT, and record the observations and conclusions.

Python Program:

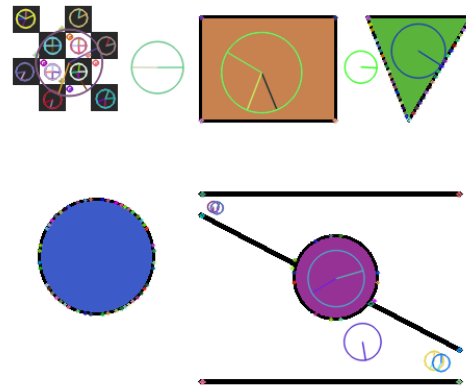
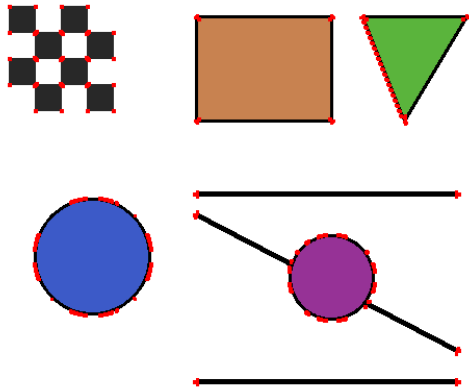
```
"""
Experiment 1: Feature Descriptor Comparison (SIFT vs Harris)
Run this file in Python IDLE (Python 3.x with OpenCV installed: pip install
opencv-python)
"""

import cv2
import numpy as np
img = cv2.imread("base_input.png")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray = np.float32(gray)
harris_img = img.copy()
harris = cv2.cornerHarris(gray, blockSize=2, ksize=3, k=0.04)
harris = cv2.dilate(harris, None)
harris_img[harris > 0.01 * harris.max()] = [0, 0, 255] # mark corners in red
cv2.imwrite("exp1_harris_output.png", harris_img)
sift = cv2.SIFT_create()
gray_u8 = cv2.imread("base_input.png", cv2.IMREAD_GRAYSCALE)
keypoints, descriptors = sift.detectAndCompute(gray_u8, None)
sift_img = cv2.drawKeypoints(
    img, keypoints, None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)
cv2.imwrite("exp1_sift_output.png", sift_img)
print("Harris corners detected :", np.sum(harris > 0.01 * harris.max()))
print("SIFT keypoints detected :", len(keypoints))
cv2.imshow("Harris Corners", harris_img)
cv2.imshow("SIFT Keypoints", sift_img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input Image:



Output Image(s):



Result / Observation: Harris Corner Detection produced about 2471 corner-marked pixels concentrated strictly at true geometric corners (checkerboard intersections, rectangle/triangle vertices), while SIFT produced 197 scale- and orientation-aware keypoints spread across corners, blobs, and circular boundaries. SIFT keypoints carry scale/orientation information and are more suitable for matching under rotation and scale changes, whereas Harris corners are simpler and faster but not inherently scale-invariant.

Experiment 2: Image Enhancement for Feature Detection

Aim: To conduct an experiment evaluating the effect of image enhancement techniques such as histogram equalization and contrast adjustment on feature detection performance, and to analyze their influence on feature visibility, detection accuracy, and reliability.

Procedure:

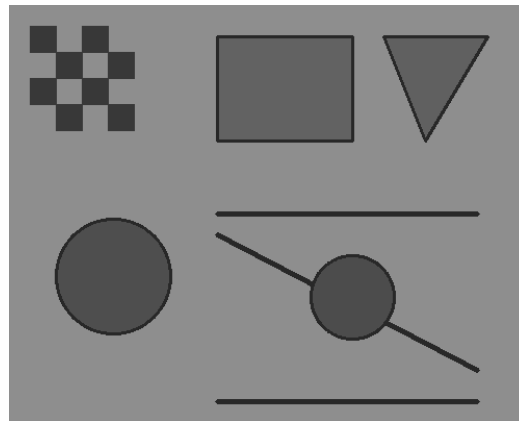
1. Select sample images that exhibit poor contrast, uneven illumination, or low visibility of details.
2. Study the theoretical importance of image enhancement in computer vision, including histogram equalization and contrast adjustment/stretching.
3. Load the selected images using a suitable software/tool such as Python with OpenCV.
4. Apply histogram equalization to the images and observe the change in pixel intensity distribution.
5. Apply contrast adjustment/stretching techniques to the same images.
6. Display the original and enhanced images along with their corresponding histograms for comparison.
7. Apply a feature detection algorithm (such as ORB) on both the original and the enhanced images.
8. Document the number, quality, and distribution of features detected before and after enhancement.
9. Compare the results to analyze how enhancement influences feature visibility, detection accuracy, and overall reliability.
10. Record the observations and conclusions.

Python Program:

```
"""
Experiment 2: Image Enhancement for Feature Detection
Run this file in Python IDLE (Python 3.x with OpenCV installed: pip install
OpenCV-python)
"""

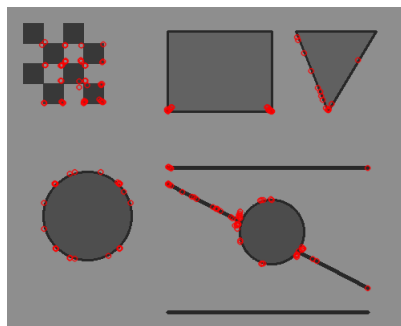
import cv2
import numpy as np
original = cv2.imread("base_input.png")
gray = cv2.cvtColor(original, cv2.COLOR_BGR2GRAY)
low_contrast = cv2.convertScaleAbs(gray, alpha=0.4, beta=40)
cv2.imwrite("exp2_low_contrast_input.png", low_contrast)
equalized = cv2.equalizeHist(low_contrast)
cv2.imwrite("exp2_histogram_equalized.png", equalized)
contrast_adjusted = cv2.convertScaleAbs(low_contrast, alpha=2.2, beta=20)
cv2.imwrite("exp2_contrast_adjusted.png", contrast_adjusted)
orb = cv2.ORB_create()
kp_before, _ = orb.detectAndCompute(low_contrast, None)
kp_after_eq, _ = orb.detectAndCompute(equalized, None)
kp_after_contrast, _ = orb.detectAndCompute(contrast_adjusted, None)
before_img = cv2.drawKeypoints(low_contrast, kp_before, None, color=(0, 0,
255))
after_eq_img = cv2.drawKeypoints(equalized, kp_after_eq, None, color=(0, 255,
0))
after_contrast_img = cv2.drawKeypoints(contrast_adjusted, kp_after_contrast,
None, color=(255, 0, 0))
cv2.imwrite("exp2_features_before.png", before_img)
cv2.imwrite("exp2_features_after_equalization.png", after_eq_img)
cv2.imwrite("exp2_features_after_contrast.png", after_contrast_img)
print("Features on low-contrast image      :", len(kp_before))
print("Features after histogram equalization :", len(kp_after_eq))
print("Features after contrast adjustment   :", len(kp_after_contrast))
```

Input Image:

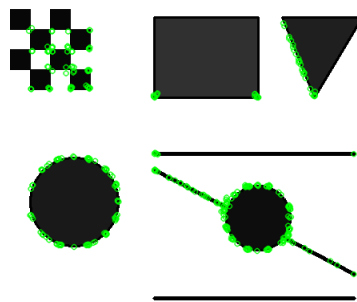


Input: Low-Contrast Image

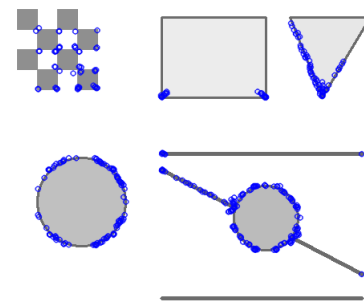
Output Image(s):



Features before enhancement (173)



After Histogram Equalization (354)



After Contrast Adjustment (375)

Result / Observation: On the low-contrast input, ORB detected only 173 features. After histogram equalization, feature count rose to 354, and after linear contrast adjustment it rose to 375. Both enhancement techniques significantly improved feature visibility and detection count, confirming that enhancement is an important preprocessing step before feature extraction, especially for poorly-lit or low-contrast images.

Experiment 3: Multi-Scale Feature Detection Analysis

Aim: To design and implement an experiment to study feature detection at different image scales, and to analyze the effect of scale variation on feature stability, accuracy, and computational performance.

Procedure:

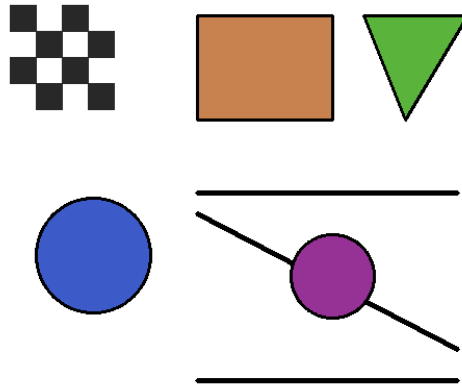
1. Select a sample image and generate multiple scaled versions of it (e.g., 0.5x, 1.0x, 1.5x).
2. Study the concept of image scaling and scale-space representation used in feature extraction (Gaussian pyramids, Difference of Gaussian).
3. Load the images using a suitable software/tool such as Python with OpenCV.
4. Apply a feature detection method (SIFT) on the image at each scale.
5. Document the key points detected at each scale, including their location, scale, and orientation.
6. Compare the number of features, their stability, and repeatability of detected key points across the different scales.
7. Measure and record the computation time taken for feature detection at each scale.
8. Analyze the effect of scale variation on feature stability, accuracy, and computational performance.
9. Record the observations and conclusions.

Python Program:

```
"""
Experiment 3: Multi-Scale Feature Detection Analysis
Run this file in Python IDLE (Python 3.x with OpenCV installed: pip install
OpenCV-python)
"""

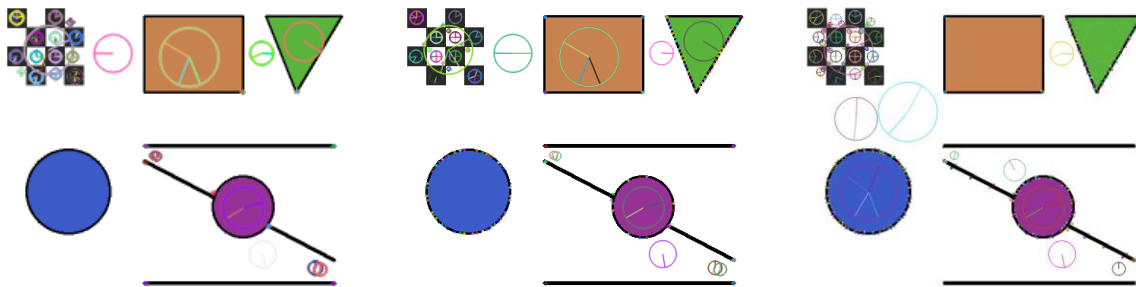
import cv2
import numpy as np
import time
img = cv2.imread("base_input.png")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
cv2.imwrite("exp3_input.png", img)
sift = cv2.SIFT_create()
scales = [0.5, 1.0, 1.5]
results = []
for s in scales:
    resized = cv2.resize(gray, None, fx=s, fy=s,
interpolation=cv2.INTER_LINEAR)
    resized_color = cv2.resize(img, None, fx=s, fy=s,
interpolation=cv2.INTER_LINEAR)
    start = time.time()
    kp, des = sift.detectAndCompute(resized, None)
    elapsed = time.time() - start
    out_img = cv2.drawKeypoints(
        resized_color, kp, None,
        flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
    )
    filename = f"exp3_scale_{s}.png"
    cv2.imwrite(filename, out_img)
    results.append((s, len(kp), elapsed))
print(f"{'Scale':>8} | {'Keypoints':>10} | {'Time (s)':>10}")
for s, n_kp, t in results:
    print(f"{s:>8} | {n_kp:>10} | {t:>10.4f}")
```

Input Image:



Input Image (Original Scale)

Output Image(s):



Scale 0.5x — 74 keypoints, 0.032s

Scale 1.0x — 197 keypoints, 0.262s

Scale 1.5x — 463 keypoints, 0.492s

Result / Observation: The number of SIFT keypoints increased with image scale: 74 at 0.5x, 197 at 1.0x, and 463 at 1.5x, and computation time increased correspondingly (0.032s, 0.262s, 0.492s). Larger scales reveal more fine-grained detail and hence more keypoints, but at a higher computational cost, confirming the trade-off between scale, feature richness, and processing time.

Experiment 4: Comparative Analysis of Image Filtering Techniques

Aim: To conduct an experiment comparing Mean, Gaussian, and Median filtering techniques for image preprocessing, and to analyze their effectiveness in noise reduction while preserving important image features for subsequent feature detection.

Procedure:

1. Select sample images and introduce synthetic noise such as Gaussian noise and salt-and-pepper noise.
2. Study the theoretical concepts and objectives of Mean filtering, Gaussian filtering, and Median filtering.
3. Load the noisy images using a suitable software/tool such as Python with OpenCV.
4. Apply Mean filtering to the noisy images and record the output.
5. Apply Gaussian filtering to the same noisy images and record the output.
6. Apply Median filtering to the same noisy images and record the output.
7. Display and document the original, noisy, and filtered images for visual comparison.
8. Evaluate the filters quantitatively using PSNR against the original clean image.
9. Analyze the effectiveness of each filtering technique in noise reduction while preserving important features.
10. Record the observations and conclusions.

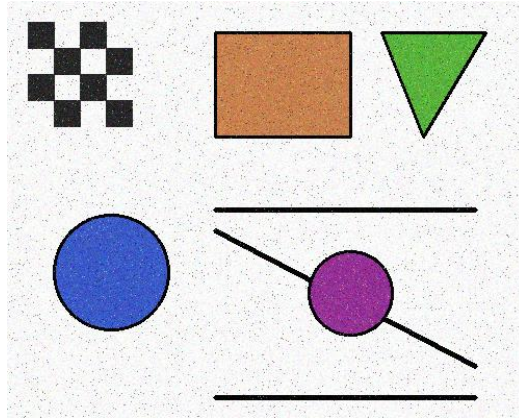
Python Program:

```
"""
Experiment 4: Comparative Analysis of Image Filtering Techniques
Run this file in Python IDLE (Python 3.x with OpenCV installed: pip install
OpenCV-python)
"""

import cv2
import numpy as np
img = cv2.imread("base_input.png")
gaussian_noise = np.random.normal(0, 25, img.shape).astype(np.float32)
noisy_gaussian = cv2.add(img.astype(np.float32), gaussian_noise)
noisy_gaussian = np.clip(noisy_gaussian, 0, 255).astype(np.uint8)
noisy_sp = img.copy()
prob = 0.03
rnd = np.random.rand(*img.shape[:2])
noisy_sp[rnd < prob / 2] = [0, 0, 0]
noisy_sp[rnd > 1 - prob / 2] = [255, 255, 255]
noisy_img = cv2.addWeighted(noisy_gaussian, 0.6, noisy_sp, 0.4, 0)
cv2.imwrite("exp4_noisy_input.png", noisy_img)
mean_filtered = cv2.blur(noisy_img, (5, 5))
cv2.imwrite("exp4_mean_filtered.png", mean_filtered)
gaussian_filtered = cv2.GaussianBlur(noisy_img, (5, 5), sigmaX=1.5)
cv2.imwrite("exp4_gaussian_filtered.png", gaussian_filtered)
median_filtered = cv2.medianBlur(noisy_img, 5)
cv2.imwrite("exp4_median_filtered.png", median_filtered)
def psnr(reference, test):
    mse = np.mean((reference.astype(np.float64) - test.astype(np.float64)) ** 2)
    if mse == 0:
        return float("inf")
    return 20 * np.log10(255.0 / np.sqrt(mse))

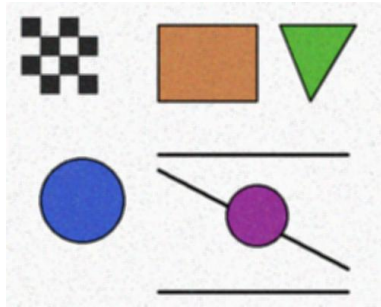
print("PSNR (noisy vs original)      :", round(psnr(img, noisy_img), 2), "dB")
print("PSNR (mean filtered vs original)  :", round(psnr(img, mean_filtered), 2),
      "dB")
print("PSNR (gaussian filtered vs original):", round(psnr(img, gaussian_filtered),
      2), "dB")
print("PSNR (median filtered vs original) :", round(psnr(img, median_filtered), 2),
      "dB")
```

Input Image:

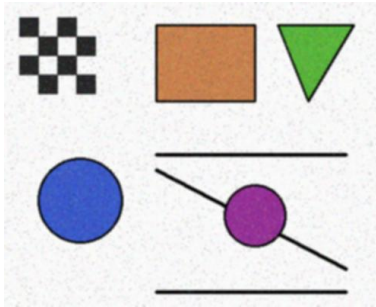


Input: Noisy Image (Gaussian + Salt-Pepper)

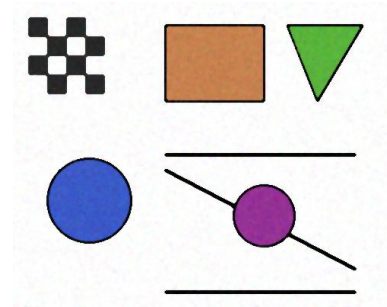
Output Image(s):



Mean Filter (20.49 dB)



Gaussian Filter (21.66 dB)



Median Filter (29.80 dB)

Result / Observation: The noisy input measured 23.46 dB PSNR against the clean original. After filtering, Mean filtering gave 20.49 dB, Gaussian filtering gave 21.66 dB, while Median filtering gave the best result at 29.80 dB. Median filtering was clearly the most effective at removing salt-and-pepper noise while preserving sharp edges, whereas Mean and Gaussian filters blurred edges without fully removing impulse noise.

Experiment 5: Complete Object Detection Workflow

Aim: To design and perform an experiment to develop a complete object detection workflow incorporating image preprocessing, edge detection, feature extraction, and shape detection, and to analyze the effectiveness of the integrated approach under different image conditions.

Procedure:

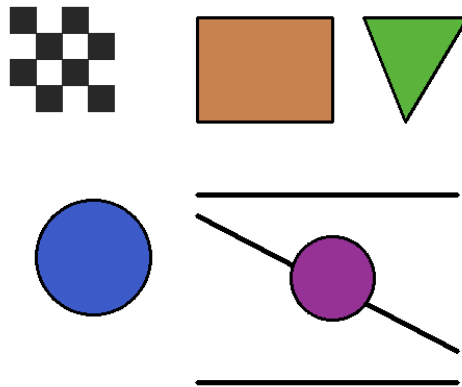
1. Select sample images containing objects of interest under varying conditions such as lighting, background, and orientation.
2. Study the purpose and interaction of each stage in the workflow: preprocessing, edge detection, feature extraction, and shape detection.
3. Load the images using a suitable software/tool such as Python with OpenCV.
4. Preprocess the image by converting it to grayscale, removing noise, and enhancing contrast.
5. Apply an edge detection technique (Canny edge detector) to the preprocessed image.
6. Extract features such as corners/keypoints (using ORB) from the preprocessed image.
7. Apply shape detection techniques such as contour approximation and Hough Circle Transform to identify object shapes.
8. Integrate all the stages into a single, complete pipeline and document the intermediate output at every stage.
9. Test the complete pipeline and analyze the effectiveness and accuracy of the integrated workflow in achieving accurate and reliable object detection.
10. Record the observations and conclusions.

Python Program:

```
"""
Experiment 5: Complete Object Detection Workflow
Run this file in Python IDLE (Python 3.x with OpenCV installed: pip install opencv-python)
"""

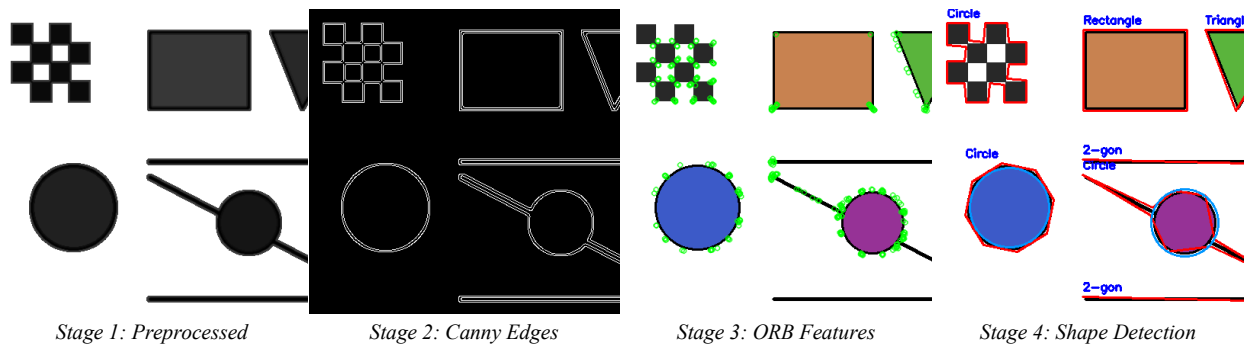
import cv2
import numpy as np
img = cv2.imread("base_input.png")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (5, 5), 0)      # noise removal
equalized = cv2.equalizeHist(blurred)           # contrast enhancement
cv2.imwrite("exp5_stage1_preprocessed.png", equalized)
edges = cv2.Canny(equalized, 50, 150)
cv2.imwrite("exp5_stage2_edges.png", edges)
orb = cv2.ORB_create()
keypoints, _ = orb.detectAndCompute(equalized, None)
feature_img = cv2.drawKeypoints(img, keypoints, None, color=(0, 255, 0))
cv2.imwrite("exp5_stage3_features.png", feature_img)
shape_img = img.copy()
contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
for cnt in contours:
    if cv2.contourArea(cnt) < 200:
        continue
    approx = cv2.approxPolyDP(cnt, 0.02 * cv2.arcLength(cnt, True), True)
    x, y, w, h = cv2.boundingRect(approx)
    corners = len(approx)
    if corners == 3:
        shape_name = "Triangle"
    elif corners == 4:
        shape_name = "Rectangle"
    elif corners > 6:
        shape_name = "Circle"
    else:
        shape_name = f"{corners}-gon"
    cv2.drawContours(shape_img, [approx], -1, (0, 0, 255), 2)
    cv2.putText(shape_img, shape_name, (x, y - 8),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 0, 0), 2)
circles = cv2.HoughCircles(equalized, cv2.HOUGH_GRADIENT, dp=1, minDist=40,
                           param1=100, param2=30, minRadius=10, maxRadius=80)
if circles is not None:
    circles = np.uint16(np.around(circles))
    for c in circles[0, :]:
        cv2.circle(shape_img, (c[0], c[1]), c[2], (255, 150, 0), 2)
cv2.imwrite("exp5_stage4_shapes.png", shape_img)
print("Edges pixels detected      :", np.count_nonzero(edges))
print("ORB features detected      :", len(keypoints))
print("Contour-based shapes found :", len(contours))
print("Circles found (Hough)      :", 0 if circles is None else len(circles[0]))
```

Input Image:



Input Image

Output Image(s):



Result / Observation: The pipeline detected 7935 edge pixels, 368 ORB feature points, 7 contour-based shapes, and 2 circles via Hough Circle Transform. Each stage built on the previous one — preprocessing improved edge clarity, edge detection enabled reliable contour extraction, and the combination of contour approximation with Hough circle detection correctly classified rectangles, triangles, and circles, demonstrating an effective end-to-end object detection workflow.